

Contents

Amazon Beauty Reviews — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	2
Your manager's request	3
Your team's job for the next 2 weeks (Module 3)	3
After Module 3	3
How This Guide Works (Read This First!)	4
We do NOT give you the full code	4
Visualizations — Two Modes	4
1. Exploratory charts (for YOU)	4
2. Explanatory charts (for DANIEL)	4
Your plotting toolkit (you learned this in Module 2 Class 5)	4
Before You Start — Google Colab Setup	5
Step 1 — Open a new Colab notebook	5
Step 2 — Connect to Google Drive	5
Step 3 — Create a project folder	5
Step 4 — Get the data	5
Step 5 — Test it	6
Colab tips	6
Class 1 — Data Cleaning	6
Your goal	6
Inputs	6
Outputs	7
Phase A — Explore the data first (15 minutes)	7
Phase B — Clean the reviews table (45 minutes)	8
Phase C — Make ONE chart for Daniel (15 minutes)	9
Common mistakes in Class 1	10
Self-check before Class 2	10
Class 2 — Encoding and Scaling	10
Your goal	10
Inputs	10
Outputs	10
Phase A — Explore the data first	11
Phase B — Encode and scale	11
Phase C — Make ONE chart for Daniel	12
Common mistakes in Class 2	13
Self-check before Class 3	13
Class 3 — Feature Engineering	13
Your goal	13
Inputs	13
Outputs	13
Phase A — Explore the data first	13
Phase B — Engineer the features	14
Phase C — Make ONE chart for Daniel	16
Common mistakes in Class 3	16
Self-check before Class 4	16
Class 4 — Feature Selection	16

Your goal	16
Inputs	17
Outputs	17
Phase A — Explore the data first	17
Phase B — Select features	17
Phase C — Make ONE chart for Daniel	19
Common mistakes in Class 4	19
Self-check before Class 5	19
Class 5 — Pipelines	19
Your goal	19
Inputs	19
Outputs	20
Phase A — Explore the data first	20
Phase B — Build the pipeline	20
Phase C — Make ONE chart for Daniel	21
Common mistakes in Class 5	22
Self-check before Class 6	22
Class 6 — End-to-End Lab (THE BIG ONE)	22
Your goal	22
Time	22
Inputs	22
Outputs	22
Phase A — Explore the data first (10 minutes)	23
Phase B — Build the final dataset (70 minutes)	23
Phase C — Findings report for Daniel (10 minutes)	24
Grading rubric (100 points)	24
Submit	25
Tips for the whole module	25

Amazon Beauty Reviews — Module 3 Lab Guide

Scenario: Amazon Beauty product reviews. Predict if a review is “helpful”. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **Amazon**. You join the **Customer Reviews Quality Team**. This team takes care of the reviews under every product page.

Amazon sells beauty products: shampoo, cream, lipstick, soap. Every product has a page. Under the page, there is a long list of reviews. Some products have **hundreds of reviews**.

Every day: - Shoppers open a product page. - They want to read 3 or 4 reviews before they decide to buy. - They do NOT have time to read 200 reviews. - So Amazon must show them the **best** reviews FIRST.

The problem

Not every review is good. Look at these examples:

“This shampoo changed my hair. I have curly hair and it was always dry. After 2 weeks of using this product, my hair is soft and shiny. The smell is light, not too strong. I will buy again. 5 stars.”

vs.

“good”

vs.

“ITEM ARRIVED BROKEN!!! WORST COMPANY!!! NEVER AGAIN!!!”

The first one is **helpful**. It tells the shopper everything. The second one is useless. The third one is angry, but does not say much about the product.

Amazon has a button under each review: “**Was this review helpful?**” Other shoppers click “Yes” or “No”. The number of “Yes” clicks is called the **vote** count.

The problem: - About **85% of reviews have 0 votes**. Nobody clicked the button. So we do not know. - Only **~15% of reviews have 5 or more “Yes” votes**. These are the proven helpful ones. - A new review starts at 0 votes. It is **buried** under old reviews. Nobody sees it. Nobody votes.

This is a chicken-and-egg problem. New helpful reviews never get seen.

Your manager’s request

Your manager, **Daniel** (Customer Reviews Quality Team Lead), tells you:

“Each product has hundreds of reviews. Some help shoppers decide. Some are useless. Some are angry spam.

Build me a model that **ranks reviews**. Surface the helpful ones at the top. Hide the spam at the bottom.

We have a helpfulness vote count, but most reviews have 0 votes because they are buried. So we cannot use votes alone. We must **predict** which new reviews will become helpful, even before anyone votes.

Then we will do TWO things: 1. Show the predicted-helpful reviews at the TOP of the product page. 2. Hide the predicted-spam reviews behind a ‘show more’ link.

If we do this right, shoppers spend less time reading junk. They decide faster. They buy more. Everyone wins.”

Your team’s job for the next 2 weeks (Module 3)

Daniel cannot do this alone. The raw data is a **big gzipped JSON file** with 370,000 reviews. It needs a lot of cleaning.

Your job in Module 3: > **Turn one messy JSON file into ONE clean parquet file. The clean file will be used to train the model in Module 4.**

The clean file is called `amazon_beauty_clean.parquet`. It must have **22 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the prediction model. Daniel finally gets his “helpful-score”.
Module 5	Find groups of reviewers (super-reviewers vs one-time buyers vs angry posters).
Module 7	Read the review text. Find common topics: “smell”, “price”, “delivery”. Use NLP.

You use the **same Amazon Beauty dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ____ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

2. Explanatory charts (for DANIEL)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**Daniel, look at this. This is what is going on with our reviews.**”

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	<code>plt.hist()</code> or <code>sns.histplot()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	See outliers in a numeric column	<code>sns.boxplot()</code>

Plot	When to use it	Function name
Scatter plot	See the relationship between 2 numbers	<code>plt.scatter()</code> or <code>sns.scatterplot()</code>
Heatmap	See correlation between many columns	<code>sns.heatmap(df.corr())</code>
Line plot	See change over time	<code>plt.plot()</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `amazon_beauty_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/amazon_beauty_lab', exist_ok=True)
%cd /content/drive/MyDrive/amazon_beauty_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

The dataset is the **Amazon Customer Reviews — Beauty 5-core** from Stanford SNAP. - “5-core” means: every reviewer wrote at least 5 reviews, every product got at least 5 reviews. Good for ML. - Source page: <https://nijianmo.github.io/amazon/index.html> - The file is **gzipped JSON**. Each line is one review (JSON object). - License: academic / non-commercial use only.

Option A — Direct download (recommended):

In Colab, run:

```
!wget -O all_beauty_5.json.gz https://snap.stanford.edu/data/amazon_v2/categoryFilesSmall/All_Beau
!ls -la all_beauty_5.json.gz
```

You should see a file around **30–50 MB**. No Kaggle account needed.

Option B — Upload by hand:

1. Download `All_Beauty_5.json.gz` on your laptop from the URL above.
2. In Colab's file panel (left sidebar), upload it.
3. Move it to your Drive folder.

Step 5 — Test it

```
import pandas as pd
df = pd.read_json('all_beauty_5.json.gz', lines=True, compression='gzip')
print(df.shape)
print(df.columns.tolist())
```

- `lines=True` tells pandas: “each line is one JSON, not the whole file is one JSON”.
- `compression='gzip'` tells pandas: “unzip on the fly”.

Should print something like `(370000, 9)` and 9 column names like `overall`, `verified`, `reviewTime`, `reviewerID`, `asin`, `style`, `reviewerName`, `reviewText`, `summary`, `unixReviewTime`, `vote`, `image`. You are ready.

Memory tip: This file is big. If Colab runs out of RAM, take a **sample** of 100,000 rows: `df = df.sample(n=100000, random_state=42).reset_index(drop=True)` This makes everything faster and the lessons are the same.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate <code>.parquet</code> files to Drive	Files in <code>/content/</code> disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime > Restart runtime.
Use <code>df.sample(100000)</code> if RAM is tight	The dataset is big. A sample teaches the same lesson.

Class 1 — Data Cleaning

Scenario reminder: Daniel drops a 50 MB gzipped JSON file on your desk. It has 370,000 beauty product reviews. Some reviews are empty. Some have weird date formats. The vote column has many missing values. Your job today: load the data, fix it, save a clean version.

Your goal

Load the gzipped JSON. Understand each column. Drop empty reviews. Fix the vote column. Save a clean parquet file.

Inputs

- `all_beauty_5.json.gz` (one big gzipped JSON file)

Outputs

- reviews_step1.parquet saved in your Drive folder
 - 3+ exploratory charts in your notebook
 - 1 explanatory chart for Daniel
 - Notes in markdown about every decision you made
-

Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data.

Exploratory chart 1 — Distribution of overall (star rating)

- **Question:** “Are most reviews 5 stars? Or are they spread across 1–5?”
- **HINTS:**
 - Use `df['overall'].value_counts().sort_index().plot.bar()`.
- **What you learn:** Amazon reviews are very skewed to 5 stars. Most people only post when they LOVE the product. 1-star reviews are rare but loud.

Exploratory chart 2 — Missing values per column

- **Question:** “Which columns have the most missing data?”
- **HINTS:**
 - Use `df.isna().sum().sort_values(ascending=False)`.
 - Plot it as a bar chart.
- **What you learn:** vote and image are mostly missing. reviewText has some empty rows. summary is almost always there.

Exploratory chart 3 — Distribution of vote (the helpfulness count)

- **Question:** “How many reviews got 0 votes? 1 vote? 10 votes? 100 votes?”
- **HINTS:**
 - First fill missing vote with 0 (do this in Phase B Step 4 anyway, do it here too as a copy).
 - The vote column is stored as a string with commas (like “1,234”). Convert with `.str.replace(',', '').astype(float)`.
 - Use a histogram with `bins=50`.
 - Or use `df['vote'].clip(upper=20).value_counts().sort_index().plot.bar()` to see 0–20 only.
- **What you learn:** A HUGE spike at 0. A long tail of “viral” reviews with 100+ votes. This is the imbalance we will deal with later.

Exploratory chart 4 — Reviews per year

- **Question:** “When were these reviews written? Did Amazon collect more reviews over time?”
 - **HINTS:**
 - Convert `unixReviewTime` to `datetime` (you do this in Phase B Step 3 anyway, do it here too).
 - Use `.dt.year`.
 - `value_counts().sort_index().plot.bar()`.
 - **What you learn:** Most reviews are from 2014–2018. Old reviews are rare.
-

Phase B — Clean the reviews table (45 minutes)

Step 1 — Load the gzipped JSON

- **WHAT:** Read the `.json.gz` file into a DataFrame.
- **HINTS:**
 - `pd.read_json('all_beauty_5.json.gz', lines=True, compression='gzip')`.
 - `lines=True` is REQUIRED for this file (one JSON per line).
- **EXPECTED:** A DataFrame with ~370,000 rows × 9–12 columns.

Step 2 — Look at the DataFrame

- **WHAT:** Check `.shape`, `.info()`, `.head()`, and `.describe()`.
- **HINTS:**
 - Look at the Dtype column in `.info()` output.
 - **Is `unixReviewTime` stored as integer?** (Yes — it is a Unix timestamp = seconds since 1970.)
 - **Is `vote` stored as text (object)?** (Yes — and some values have commas.)
- **EXPECTED:**

Column	What it is	Approx dtype
overall	Star rating 1–5	float
verified	True if purchase verified	bool
reviewTime	Date as string “MM DD, YYYY”	object
reviewerID	Reviewer ID	object
asin	Product ID	object
reviewerName	Reviewer name	object
reviewText	The full review body	object
summary	Review title	object
unixReviewTime	Date as integer	int
vote	Helpfulness votes	object (with commas)
style	Optional product variant info	object
image	URLs of uploaded images	object

Step 3 — Convert the date column

- **WHAT:** Convert `unixReviewTime` to a real datetime.
- **HINTS:**
 - The function is `pd.to_datetime()`.
 - Add the argument `unit='s'`. This tells pandas the integer is in seconds.
 - Save the result in a new column: `df['review_date'] = pd.to_datetime(df['unixReviewTime'], unit=___)`.
- **WHY:** If you only have an integer like 1391040000, you cannot compute “how old is this review?”. After conversion you get a real date like 2014-01-30.
- **EXPECTED:** `df['review_date'].dtypes` shows `datetime64[ns]`. The min year is around 2000, max around 2018.

Step 4 — Fix the vote column

- **WHAT:** `vote` has 3 problems: it has commas in numbers, it is text not number, and most cells are missing.
- **HINTS:**
 - Remove commas: `df['vote'] = df['vote'].str.replace(',', '')`.
 - Convert to number: `.astype(float)`.
 - Fill missing with 0: `.fillna(___)`.
 - Combine into one chain.
- **WHY:** Missing vote does NOT mean “no info”. It means “nobody clicked yet”. For our analysis that is the same as 0 votes.

- **EXPECTED:** `df['vote'].isna().sum() == 0`. The mean is small (under 1). The max is large (hundreds).

Step 5 — Drop empty review texts

- **WHAT:** Some rows have no reviewText. We cannot use them for NLP in Module 7.
- **HINTS:**
 - First: `df['reviewText'] = df['reviewText'].fillna('')` to make missing into empty string.
 - Then: `df = df[df['reviewText'].str.strip() != '']`.
 - Add `.copy()` at the end. Avoids a warning later.
- **WHY:** A review with no text has no information. The shopper sees nothing.
- **EXPECTED:** About **350,000 rows** left (about 95% kept). Write the exact number in markdown.

Step 6 — Fill missing summaries

- **WHAT:** Some rows have no summary (the title). Fill with empty string.
- **HINTS:**
 - `df['summary'] = df['summary'].fillna('')`.
- **WHY:** Empty string is safer than missing. Later we will count summary length, and `len('')` is 0, but `len(None)` crashes.

Step 7 — Convert verified to int

- **WHAT:** verified is True/False. Models prefer 1/0.
- **HINTS:**
 - `df['is_verified'] = df['verified'].astype(int)`.
- **WHY:** “Verified purchase” is a STRONG signal. A verified review is more trusted by other shoppers.

Step 8 — Write down what you did

In a markdown cell, write: - Starting rows: ~370,000 - After drop-empty-text: ~350,000 - vote filled with 0 for missing - verified converted to 0/1 - unixReviewTime converted to datetime - WHY you removed each group.

Step 9 — Save to Drive

- **WHAT:** Save the cleaned DataFrame as parquet, inside your Drive folder.
- **HINTS:** `df.to_parquet('/content/drive/MyDrive/amazon_beauty_lab/reviews_step1.parquet')`.

Phase C — Make ONE chart for Daniel (15 minutes)

He does not have time to read your code. He wants ONE picture.

Daniel’s chart — “Most reviews never get any votes”

A bar chart showing how many reviews fall into each “vote count” bucket:

Vote bucket	Approximate count
0 votes	very big
1–4 votes	medium
5–9 votes	small
10–49 votes	very small
50+ votes	tiny

- **HINTS:**

- Use `pd.cut()` with the bins above on the vote column.
- `value_counts().sort_index().plot.bar()`.
- Title: "Most reviews never get any votes – 85% are buried at 0".
- X-label: vote bucket.
- Y-label: number of reviews.
- Put the percentage on top of each bar.
- **Takeaway for Daniel:** "We cannot use votes alone to find helpful reviews. Most reviews never get the chance. We must PREDICT helpfulness from the review text and meta-data."

Common mistakes in Class 1

Mistake	What goes wrong
Forget <code>lines=True</code> in <code>read_json</code>	Code crashes: "trailing data".
Forget <code>unit='s'</code> in <code>to_datetime</code>	Dates come out in year 1970.
Forget to remove commas from vote	<code>astype(float)</code> crashes on "1,234".
Drop rows with missing vote instead of filling with 0	You lose 85% of the data.
Save to <code>/content/</code> instead of Drive	File disappears when Colab disconnects.

Self-check before Class 2

- ☐ DataFrame loaded from `.json.gz` (around 370,000 rows).
 - ☐ `unixReviewTime` converted to `datetime`.
 - ☐ vote is numeric with no missing values.
 - ☐ Empty `reviewText` rows dropped.
 - ☐ `is_verified` is 0/1 integer.
 - ☐ At least 3 exploratory charts in your notebook.
 - ☐ 1 polished chart for Daniel.
 - ☐ You saved `reviews_step1.parquet` to your Drive folder.
 - ☐ You wrote down (in markdown) what you did and WHY.
-

Class 2 — Encoding and Scaling

Scenario reminder: Daniel looks at your cleaned data. He is happy. But he says: "The model is a math model. It does not understand the word 'verified' as a concept, or know what 'lipstick' means. Turn the categories into numbers. And the vote column has a huge tail — squash it."

Your goal

Turn TEXT/CATEGORY columns into numbers. Make wide numeric columns smaller and friendlier for the model.

Inputs

- `reviews_step1.parquet` from Class 1

Outputs

- `reviews_step2.parquet` in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Daniel
-

Phase A — Explore the data first

Exploratory chart 1 — Distribution of vote

- **Question:** “Most votes are 0 or 1. A few reviews have 500+ votes. Is the distribution very skewed?”
- **HINTS:**
 - Histogram (`plt.hist()`) with `bins=50`.
 - You will see a huge bar at 0 and almost nothing else.
- **What you learn:** A typical “long-tail” distribution. We MUST log-transform.

Exploratory chart 2 — Distribution of vote AFTER `np.log1p`

- **Question:** “Does log-transform make the shape more readable?”
- **HINTS:**
 - Make a NEW column: `vote_log = np.log1p(df['vote'])`.
 - Histogram it.
 - Compare to chart 1.
- **What you learn:** Now the shape has multiple peaks. Much friendlier for the model.

Exploratory chart 3 — overall star rating counts

- **Question:** “How many 1-star, 2-star, 3-star, 4-star, 5-star reviews?”
- **HINTS:** `df['overall'].value_counts().sort_index().plot.bar()`.
- **What you learn:** Around 60–70% are 5 stars. Star rating is very skewed.

Exploratory chart 4 — is_verified counts

- **Question:** “What % of reviews are from verified purchasers?”
 - **HINTS:** `df['is_verified'].value_counts(normalize=True).plot.bar()`.
 - **What you learn:** Around 85–90% are verified. Verified is the norm, unverified is rare.
-

Phase B — Encode and scale

Step 1 — Decide which columns are categorical

- **WHAT:** Look at the data again. Identify the columns that are categories.
- **HINTS:** Make a short list. Candidates:
 - overall (1–5 stars) — could be category OR number.
 - is_verified (already 0/1) — done.
 - Year of review (we will create this in Step 3).
- **WRITE IN NOTEBOOK:** Which columns are numeric, which are categorical? Why?

Step 2 — One-hot encode overall (optional)

- **WHAT:** Turn the 5 star levels into 5 new 0/1 columns.
- **HINTS:**
 - Use `pd.get_dummies(df, columns=['overall'], prefix='star')`.
 - Result: `star_1, star_2, star_3, star_4, star_5`.
- **WHY one-hot here?** Star ratings could be treated as numbers OR categories. As one-hot, the model can learn “1-star reviews are different from 5-star reviews” without assuming a smooth scale.
- **CHOICE:** Some teams keep overall as a number. Some one-hot. Pick one and write WHY.

Step 3 — Extract date features

- **WHAT:** From `review_date`, make new columns year and month.
- **HINTS:**

- `df['year'] = df['review_date'].dt.year.`
- `df['month'] = df['review_date'].dt.month.`
- **WHY:** A review from 2010 may be on an old product. Recent reviews may be on trendy products. Year matters.

Step 4 — Log-transform vote

- **WHAT:** The vote column has a very long tail. Apply `np.log1p()`.
- **HINTS:**
 - `df['vote_log'] = np.log1p(df['vote']).`
- **WHY log1p and not log?** `log(0)` is `-inf`. `log1p(x) = log(x + 1)` is safe for zeros. Since most votes are 0, this matters a lot.
- **WHY at all?** The original vote ranges from 0 to ~1000. A model sees the 1000 as 1000x more important than 1. Log makes 0 stay 0, but 1000 becomes ~7. Same information, friendlier scale.

Step 5 — Compute review_age_days

- **WHAT:** How many days ago was this review written?
- **HINTS:**
 - Pick a reference date — e.g. the MAX date in the dataset: `ref = df['review_date'].max()`.
 - Subtract: `df['review_age_days'] = (ref - df['review_date']).dt.days.`
- **WHY:** Older reviews had more time to collect votes. A review from 2010 with 5 votes is less impressive than a review from 2018 with 5 votes.

Step 6 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
 - `from sklearn.preprocessing import StandardScaler.`
 - `scaler = StandardScaler().`
 - `scaler.fit_transform(df[numeric_cols]).`
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

Step 7 — Save

- **WHAT:** Save to Drive. `df.to_parquet('/content/drive/MyDrive/amazon_beauty_lab/reviews_step2.parquet')`
- **Save the UNSCALED version.** Scaling will happen inside the Pipeline later.

Phase C — Make ONE chart for Daniel

Daniel's chart — “Verified vs unverified: do they get the same votes?”

A box plot of `vote_log` for verified vs unverified reviewers.

- **HINTS:**
 - Use `sns.boxplot(x='is_verified', y='vote_log', data=df).`
 - Or `df.groupby('is_verified')['vote_log'].mean().plot.bar()` for an average view.
- **Title:** “Verified reviewers get more helpful votes.”
- **X-label:** Verified (0 = no, 1 = yes).
- **Y-label:** `log(votes + 1)`.
- **Takeaway for Daniel:** “Verified purchase is a strong signal. Other shoppers trust verified reviewers and click ‘helpful’ more often. We MUST keep this column.”

Common mistakes in Class 2

Mistake	What goes wrong
Scale BEFORE train/test split	Leakage. Scaler sees test data.
Use <code>np.log</code> on zero votes	<code>np.log(0) = -inf</code> . Crash. Use <code>np.log1p</code> .
One-hot encode reviewerID	50,000+ unique reviewers. Table explodes.
Save the scaled version	Class 5 will scale again inside the pipeline. Double scaling = wrong numbers.
Treat overall as text “5” not number 5	Model thinks “5” is a category, loses the order.

Self-check before Class 3

- ☐ `vote_log` exists.
- ☐ `year`, `month`, `review_age_days` exist.
- ☐ You decided what to do with overall (one-hot or keep as number).
- ☐ You did NOT one-hot encode reviewerID or asin.
- ☐ At least 3 exploratory charts.
- ☐ 1 chart for Daniel.
- ☐ `reviews_step2.parquet` saved to Drive.

Class 3 — Feature Engineering

Scenario reminder: Daniel says: “The raw columns are not enough. The MOST useful columns are not there. We must MAKE them. For example: how long is the review text? Does it ask questions? Does it shout in capital letters? Does it mention ‘photo’ or ‘image’? These are the signals of helpful vs spam.”

Your goal

Make NEW columns from the review TEXT. These will help the model predict helpfulness.

Inputs

- `reviews_step2.parquet`

Outputs

- `reviews_step3.parquet` in Drive
- 3+ exploratory charts
- 1 explanatory chart for Daniel

Phase A — Explore the data first

Exploratory chart 1 — `is_helpful` distribution

- First make the `is_helpful` column (see Step 1 below).
- **Question:** “What % of reviews are helpful (`vote >= 5`)?”
- **HINTS:** `df['is_helpful'].value_counts(normalize=True).plot.bar()`.
- **What you learn:** Around 10–15%. This is an **imbalanced** problem. We will deal with that in Class 5.

Exploratory chart 2 — Review length distribution

- After Step 2 (compute `review_length`), look at its distribution.
- **HINTS:** Histogram with `bins=50`. Use `df['review_length'].clip(upper=2000)` to see the bulk.
- **What you learn:** Most reviews are SHORT (under 200 chars). A few are very long essays (5000+).

Exploratory chart 3 — Review length vs `is_helpful`

- **Question:** “Are longer reviews more likely to be helpful?”
- **HINTS:**
 - Group `review_length` into bins using `pd.cut(df['review_length'], bins=[0, 50, 200, 500, 1000, 5000, 50000])`.
 - For each bin, compute `df.groupby(bin)['is_helpful'].mean()`.
 - Bar chart.
- **What you learn:** This is one of your MOST important features. Long reviews are usually more helpful.

Exploratory chart 4 — `is_verified` vs `is_helpful`

- **Question:** “Are verified reviews more likely to be helpful?”
 - **HINTS:**
 - `df.groupby('is_verified')['is_helpful'].mean().plot.bar()`.
 - **What you learn:** Verified is a good predictor. Daniel will love this chart.
-

Phase B — Engineer the features

Step 1 — Create the target column `is_helpful`

- **WHAT:** `is_helpful = 1` if `vote >= 5`, else 0.
- **HINTS:**
 - Compare: `df['vote'] >= 5`.
 - The result is True/False. Convert to int with `.astype(int)`.
- **WHY threshold 5?** Below 5 votes is too noisy (1 friend can click “helpful” once). 5+ votes means real shoppers found it useful.
- **EXPECTED:** About 10–15% of rows have `is_helpful = 1`. Confirm with `df['is_helpful'].mean()`.

Step 2 — Length features

New column	What it is
<code>review_length</code>	<code>len(reviewText)</code> in characters
<code>summary_length</code>	<code>len(summary)</code> in characters
<code>word_count</code>	Number of words in <code>reviewText</code>

- **HINTS:**
 - `df['review_length'] = df['reviewText'].str.len()`.
 - `df['word_count'] = df['reviewText'].str.split().str.len()`.
- **WHY:** A 5-word review like “this is good” cannot tell a shopper much. A 200-word review with details can.

Step 3 — Punctuation features

New column	What it is
<code>n_questions</code>	Count of <code>?</code> in <code>reviewText</code>
<code>n_exclamations</code>	Count of <code>!</code> in <code>reviewText</code>

- **HINTS:**
 - Use `.str.count()` with the character.
 - `df['n_questions'] = df['reviewText'].str.count(____)`.
- **WHY:** Lots of **!!!** often means an emotional, angry, or low-quality review. Lots of **?** might mean a confused reviewer asking questions instead of giving info.

Step 4 — Capital letters ratio `caps_ratio`

- **WHAT:** Fraction of letters that are UPPERCASE. A spam review like “BEST PRODUCT EVER!!! BUY NOW!!!” has `caps_ratio` close to 1.
- **HINTS:**
 - Skeleton:

```
def caps_ratio(text):
    if len(text) == 0:
        return 0.0
    n_upper = sum(1 for c in text if c.____()) # YOU fill in
    n_letters = sum(1 for c in text if c.____())
    if n_letters == 0:
        return 0.0
    return n_upper / n_letters
df['caps_ratio'] = df['reviewText'].apply(____)
```
 - Fill in: `isupper`, `isalpha`, and the function name in `apply`.
- **WHY:** Shouting reviews look like spam. Shoppers downvote them. So `caps_ratio` predicts NOT-helpful.

Step 5 — Keyword features

New column	What it is
<code>has_image_keyword</code>	1 if review mentions “photo”, “picture”, “image”, “pic”
<code>has_brand_mention</code>	1 if review mentions a known brand (e.g. “Loreal”, “Nivea”, “Maybelline”, “Dove”)

- **HINTS:**
 - Lowercase first: `text = df['reviewText'].str.lower()`.
 - Use `.str.contains('|'.join(keywords), regex=True)` then `.astype(int)`.
 - Skeleton:

```
image_words = ['photo', 'picture', ____, ____]
df['has_image_keyword'] = df['reviewText'].str.lower().str.contains(____).astype(int)
```
 - Build a small brand list for `has_brand_mention`. Even 5–10 brands are enough.
- **WHY image keyword?** Reviews that say “see my photo” often have uploaded photos. Photos are gold for shoppers. These reviews get more “helpful” clicks.
- **WHY brand mention?** A review that names the brand is usually a thoughtful, comparative review.

Step 6 — Sentiment polarity score `sentiment_polarity_score`

- **WHAT:** A number from -1 (very negative) to +1 (very positive) for each review.
- **HINTS:**
 - Use the library `TextBlob`. In Colab:

```
* !pip install textblob -q
* from textblob import TextBlob
```
 - For each review: `TextBlob(text).sentiment.polarity` returns a number.
 - Apply it: `df['sentiment_polarity_score'] = df['reviewText'].apply(lambda t: TextBlob(t).sentiment.polarity)`.
 - **SLOW WARNING:** This takes a few minutes. Be patient.
- **WHY:** Very negative reviews AND very positive reviews tend to be more helpful than neutral “meh” reviews. The polarity matters.

Step 7 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/amazon_beauty_lab/reviews_step3.parquet')`.
-

Phase C — Make ONE chart for Daniel

Daniel's chart — “Long reviews are 3x more likely to be helpful”

A bar chart: bin `review_length` into 5 buckets, show the % helpful in each bucket.

- **HINTS:**
 - `bins = [0, 50, 200, 500, 1000, 10000]`.
 - `df['length_bucket'] = pd.cut(df['review_length'], bins=___)`.
 - `helpful_rate = df.groupby('length_bucket')['is_helpful'].mean() * 100`.
 - Bar chart.
 - **Title:** “Helpful rate by review length — longer reviews win.”
 - **X-label:** Review length bucket (characters).
 - **Y-label:** % of reviews marked helpful.
 - **Takeaway for Daniel:** “If a reviewer writes more than 200 characters, the review is 3x more likely to be useful. We should remind reviewers to write longer reviews. We should also show longer reviews higher in the ranking.”
-

Common mistakes in Class 3

Mistake	What goes wrong
Use <code>vote</code> as a feature for <code>is_helpful</code>	Leakage. <code>is_helpful</code> IS derived from <code>vote</code> . The model will be 100% accurate but useless.
Forget to lowercase before keyword search	“Photo” and “photo” not matched the same.
Use <code>len</code> on a column that has missing values	Crash. Always <code>fillna('')</code> first.
Skip the <code>if len(text) == 0</code> guard in <code>caps_ratio</code>	Division by zero.
Sentiment too slow — give up	Take a sample of 50,000 first to test. Then run on full data overnight.

Self-check before Class 4

- ☐ `is_helpful` exists. Mean ~10–15%.
 - ☐ `review_length`, `summary_length`, `word_count` exist.
 - ☐ `n_questions`, `n_exclamations`, `caps_ratio` exist.
 - ☐ `has_image_keyword`, `has_brand_mention` exist.
 - ☐ `sentiment_polarity_score` exists.
 - ☐ 3+ exploratory charts.
 - ☐ 1 explanatory chart.
 - ☐ `reviews_step3.parquet` saved to Drive.
-

Class 4 — Feature Selection

Scenario reminder: Now you have ~25 columns. Daniel says: “Too many. Some say the same thing. Some are useless. I want 12–15 GOOD columns. Pick them. And tell me WHY each one is in.”

Your goal

Pick the best 12–15 columns. Drop the rest. Justify every choice.

Inputs

- reviews_step3.parquet

Outputs

- reviews_step4.parquet in Drive (only the selected columns + is_helpful)
 - 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
 - A short markdown report explaining your choices
-

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
 - Compute `df[numeric_cols].corr()`.
 - Plot with `sns.heatmap()`.
 - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** review_length and word_count will have very high correlation (~0.95). Drop one. Maybe vote_log and is_helpful are correlated too — **DANGER** — that means leakage. Drop vote_log from the feature list!

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
 - Sort the values, then bar chart.
- **What you learn:** Which columns predict is_helpful strongest. review_length, is_verified, caps_ratio should be at the top.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
 - **HINTS:** `pd.Series(rf.feature_importances_, index=feature_names).sort_values().plot.barh()`.
 - **What you learn:** A second opinion on feature importance.
-

Phase B — Select features

Step 1 — Drop the leakage column FIRST

- **WHAT:** vote and vote_log MUST NOT be used as features. is_helpful was made from vote.
- **HINTS:**
 - Build a list of “forbidden” columns: `['vote', 'vote_log']`.
 - Drop them before doing anything else.
- **WHY:** If you keep vote as a feature, the model can just check “is vote >= 5?” and be 100% right on the training set. But it will FAIL on new reviews (which have 0 votes yet).

Step 2 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
 - from `sklearn.model_selection` import `train_test_split`.
 - `X = df.drop('is_helpful', axis=1); y = df['is_helpful']`.

- Pass: `test_size=0.2, random_state=42, stratify=___`.
- Fill in `stratify=y` so the imbalance ratio is preserved.
- **WHY:** All next steps use TRAIN ONLY. Otherwise leakage.

Step 3 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same.
- **HINTS:**
 - `from sklearn.feature_selection import VarianceThreshold.`
 - `VarianceThreshold(threshold=0.01)` then `.fit_transform(X_train[numeric_cols]).`
- **EXPECTED:** Maybe `has_brand_mention` survives only if 5%+ of reviews mention a brand. If not, drop.

Step 4 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where `|corrl| > 0.9`.
- **HINTS:**
 - Compute correlation matrix.
 - Get the upper triangle with `np.triu(...)`.
 - For each column, if any of its upper-triangle correlations is `> 0.9`, mark it for dropping.
- **EXPECTED:** `review_length` and `word_count` overlap. Keep ONE.

Step 5 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about `is_helpful`.
- **HINTS:**
 - `from sklearn.feature_selection import mutual_info_classif.`
 - `mi = mutual_info_classif(X_train_numeric, y_train).`
 - Put in a Series, sort.
- **EXPECTED:** Top columns should be: `review_length`, `is_verified`, `review_age_days`, `caps_ratio`, `sentiment_polarity_score`.

Step 6 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF, look at `feature_importances_`.
- **HINTS:**
 - `from sklearn.ensemble import RandomForestClassifier.`
 - `RandomForestClassifier(n_estimators=50, max_depth=8, n_jobs=-1, class_weight='balanced').`
 - `.fit(X_train, y_train).`
 - Look at `.feature_importances_`.
- **WHY `class_weight='balanced'`?** Because only ~15% of reviews are helpful. Without `balanced`, the RF predicts “not helpful” for everyone.

Step 7 — Pick the final 12–15 columns

- **WHAT:** Combine the rankings. Pick columns that:
 - Are high in mutual info, AND
 - Are high in RF importance, AND
 - Were not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

Step 8 — Save

- **HINTS:** Keep only the selected columns + `is_helpful`. Save as `reviews_step4.parquet` to Drive.

Phase C — Make ONE chart for Daniel

Daniel's chart — “These are the 10 columns that predict helpfulness”

A horizontal bar chart of your top 10 features and their importance score.

- **HINTS:**
 - Use the RF importance.
 - `sort_values()` then `.head(10)` then `.plot.barh()`.
 - **Title:** “Top 10 predictors of a helpful review.”
 - **Y-axis:** column names.
 - **X-axis:** Random Forest importance.
 - **Takeaway for Daniel:** “Review length, verified status, and review age explain 60% of helpfulness. We do not need fancy features. The simple ones win.”
-

Common mistakes in Class 4

Mistake	What goes wrong
Keep vote or vote_log as a feature	Catastrophic leakage. Model looks perfect, fails in production.
Compute mutual info on the FULL data	Leakage. Use train only.
Drop a column without writing why	Module 4 students will not understand.
Keep 30+ columns	Slow training. Overfitting risk.
Forget <code>class_weight='balanced'</code> in RF	RF says “not helpful” to everything. All importances broken.

Self-check before Class 5

- ☐ vote and vote_log REMOVED from the features.
 - ☐ Train/test split done FIRST.
 - ☐ 12–15 columns remain + `is_helpful`.
 - ☐ You wrote down WHY for each kept column.
 - ☐ 3+ exploratory charts.
 - ☐ 1 chart for Daniel.
 - ☐ `reviews_step4.parquet` saved to Drive.
-

Class 5 — Pipelines

Scenario reminder: Daniel says: “Your cleaning code is in 4 different notebooks. When a new review is posted tomorrow, you cannot run 4 notebooks. We need ONE object that takes a raw review and outputs a helpfulness score. End-to-end.”

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code. Handle the class imbalance.

Inputs

- `reviews_step4.parquet` (selected columns)

Outputs

- amazon_beauty_pipeline.joblib saved in Drive
 - 1 confusion-matrix chart + 1 ROC curve chart
 - A pipeline that takes RAW input and produces predictions
-

Phase A — Explore the data first

Exploratory chart 1 — Confusion matrix (after training)

- After Step 6, build a confusion matrix.
- **HINTS:**
 - from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay.
 - cm = confusion_matrix(y_test, y_pred).
 - ConfusionMatrixDisplay(cm, display_labels=['Not helpful', 'Helpful']).plot().
- **What you learn:** How many helpful reviews did we CATCH? How many did we MISS? How many false alarms?

Exploratory chart 2 — ROC curve

- **HINTS:**
 - from sklearn.metrics import RocCurveDisplay.
 - RocCurveDisplay.from_estimator(pipeline, X_test, y_test).
 - **What you learn:** The trade-off between recall and false alarms. The AUC number.
-

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
 - numeric: review_length, summary_length, word_count, n_questions, n_exclamations, caps_ratio, sentiment_polarity_score, review_age_days, overall.
 - categorical: month (if you kept it), year (if you kept it).
 - binary already 0/1: is_verified, has_image_keyword, has_brand_mention. Treat as numeric.

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
 - from sklearn.pipeline import Pipeline.
 - from sklearn.impute import SimpleImputer.
 - from sklearn.preprocessing import StandardScaler.
 - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```
 - Fill in the blank: best strategy for numeric? ('median')

Step 3 — Build the categorical mini-pipeline

- **WHAT:** 2 steps: imputer + one-hot encoder.
- **HINTS:**
 - from sklearn.preprocessing import OneHotEncoder.

- Skeleton:


```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='__')),
    ('onehot', OneHotEncoder(handle_unknown='__', sparse_output=__)),
])
```
- Fill in blanks. (Strategy 'most_frequent', handle_unknown 'ignore', sparse_output False.)
- **WHY handle_unknown='ignore'?** In production, a new month or a new year may appear. We do not want the code to crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
 - from sklearn.compose import ColumnTransformer.
 - It takes a list of tuples: (name, transformer, list_of_columns).

Step 5 — Add the model on top — DEAL WITH IMBALANCE

- **HINTS:**
 - from sklearn.linear_model import LogisticRegression.
 - LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42).
 - Put it in a Pipeline with the preprocessor.
- **WHY class_weight='balanced'?** Only ~15% of reviews are helpful. Without this, the model just predicts “not helpful” for everyone. It gets 85% accuracy but is useless. With class_weight='balanced', scikit-learn gives extra weight to the rare “helpful” class. Now the model actually learns to find them.
- **ALTERNATIVE — SMOTE:** Another way to fix imbalance is to OVERSAMPLE the rare class with synthetic data. Library: imbalanced-learn. **Pick ONE method — not both.** For this lab, use class_weight='balanced'. SMOTE is more advanced.

Step 6 — Train and evaluate

- **HINTS:**
 - full_pipeline.fit(X_train, y_train).
 - y_pred = full_pipeline.predict(X_test).
 - from sklearn.metrics import classification_report.
- **EXPECTED:** F1 on the helpful class around 0.30–0.45. Recall on helpful class around 0.55–0.70. (Module 4 improves this with better models.)

Step 7 — Save the trained pipeline

- **HINTS:**
 - import joblib.
 - joblib.dump(full_pipeline, '/content/drive/MyDrive/amazon_beauty_lab/amazon_beauty_pipeline.pkl')

Phase C — Make ONE chart for Daniel

Daniel’s chart — “How many helpful reviews did we catch?”

A simple confusion matrix:

	Predicted not helpful	Predicted helpful
Actually not helpful	true negatives	false alarms
Actually helpful	missed helpful	caught helpful

- **HINTS:** Just print the confusion matrix with labels. Or use `seaborn.heatmap()` on it.

- **Title:** “Of 100 helpful reviews, our baseline model catches ~60.”
 - **Takeaway for Daniel:** “Not perfect, but much better than zero. Without the model, we have no way to find helpful new reviews. Module 4 will push this higher with XGBoost and text features.”
-

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Forget <code>sparse_output=False</code> in <code>OneHotEncoder</code>	Some downstream code breaks.
Use <code>class_weight='balanced'</code> AND SMOTE together	Over-correction. Pick one.
Forget <code>random_state</code>	Results change every run. Hard to debug.
Judge the model on accuracy alone	85% accuracy means nothing on imbalanced data. Look at F1 and recall on the rare class.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
 - ☐ Preprocessor has numeric + categorical transformers.
 - ☐ `handle_unknown='ignore'` set.
 - ☐ `class_weight='balanced'` set.
 - ☐ Confusion matrix + ROC curve charts.
 - ☐ `amazon_beauty_pipeline.joblib` saved to Drive.
-

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: Daniel is in the meeting room. He wants the FINAL clean dataset on his desk in 90 minutes. This is the lab.

Your goal

Take the raw gzipped JSON file. Produce ONE final .parquet file with the exact 22-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Inputs

- `all_beauty_5.json.gz` in your Drive folder

Outputs

- `amazon_beauty_clean.parquet` (~350,000 rows × 22 columns) in Drive
 - `findings.md` (~1 page)
 - Your full Colab notebook
-

Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1–5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. `is_helpful` distribution (the ~15% imbalance problem) 2. Review length vs helpful rate (your most important chart) 3. Top 10 most important features (from the RF)

These 3 charts tell Daniel the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

Your `amazon_beauty_clean.parquet` MUST have these 22 columns, with these names and types:

#	Column	Type	Source
1	<code>reviewerID</code>	string	raw
2	<code>asin</code>	string	raw (product id)
3	<code>overall</code>	int 1-5	raw (star rating)
4	<code>is_verified</code>	int (0/1)	from <code>verified</code>
5	<code>vote</code>	float	raw, missing filled with 0
6	<code>vote_log</code>	float	engineered (log1p of vote) — kept for analysis only, NOT a feature
7	<code>unixReviewTime</code>	int	raw
8	<code>review_date</code>	datetime	engineered from <code>unixReviewTime</code>
9	<code>year</code>	int	engineered
10	<code>month</code>	int	engineered
11	<code>review_age_days</code>	int	engineered
12	<code>review_length</code>	int	engineered (len of <code>reviewText</code>)
13	<code>summary_length</code>	int	engineered (len of summary)
14	<code>word_count</code>	int	engineered
15	<code>n_questions</code>	int	engineered
16	<code>n_exclamations</code>	int	engineered
17	<code>caps_ratio</code>	float	engineered
18	<code>has_image_keyword</code>	int (0/1)	engineered
19	<code>has_brand_mention</code>	int (0/1)	engineered
20	<code>sentiment_polarity_score</code>	float	engineered (TextBlob)
21	<code>is_helpful</code>	int (0/1)	TARGET (engineered from <code>vote >= 5</code>)
22	<code>reviewText</code>	string	raw (kept for Module 7 NLP!)

Note: `summary` (review title) can also be kept as column 23 if you have memory. Daniel will thank you in Module 7.

Lab phases (timed)

Phase	Time	What you do
1. Load	10 min	Load gzipped JSON. Confirm row count.

Phase	Time	What you do
2. Clean	15 min	Fix vote, fill missing, drop empty reviewText, convert dates.
3. Date features	10 min	year, month, review_age_days.
4. Text features	25 min	review_length, summary_length, word_count, n_questions, n_exclamations, caps_ratio, has_image_keyword, has_brand_mention, sentiment_polarity_score.
5. Target + save	10 min	Compute is_helpful. Validate schema. Save .parquet.
6. Findings	10 min	Write findings.md.

Total: 90 minutes.

Memory warning: If Colab is slow, sample 100,000 rows after Phase 1: `df = df.sample(n=100000, random_state=42).reset_index(drop=True)`. The lesson is the same.

Phase C — Findings report for Daniel (10 minutes)

Write `findings.md` with these sections:

Final numbers

- Final row count: _____
- `is_helpful` rate: _____% (should be 10–15%)
- Number of unique reviewers: _____
- Number of unique products (asin): _____

Top 3 insights

1. _____ (example: “Long reviews are 3x more helpful”)
2. _____ (example: “Verified reviewers get more ‘helpful’ clicks”)
3. _____ (example: “Reviews with high caps ratio are spam — never helpful”)

One question to investigate in Module 4

- _____ (example: “Can XGBoost beat Logistic Regression on this imbalanced dataset?”)

One chart that summarizes everything

Embed your most important chart (review length vs helpful rate).

Grading rubric (100 points)

Item	Points
All 22 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10

Item	Points
Pipeline is reproducible (re-run from raw JSON.gz in ONE command)	15
vote and vote_log are NOT used as features for is_helpful	10
is_helpful rate is between 10–15%	10
At least 3 exploratory + 1 explanatory chart per class	15
findings.md has insights, not just numbers	10
Code is clean (comments, sensible variable names)	5

Submit

Upload to module-3/class_6/submissions/<YourTeamName>/: - amazon_beauty_clean.parquet - Your Colab notebook (File > Download > Download .ipynb) - findings.md

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate .parquet files to Drive** (reviews_step1, reviews_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **NEVER use vote as a feature for is_helpful.** This is the #1 mistake in this scenario. Always remove vote and vote_log from the feature list.
8. **Watch your RAM.** This dataset is bigger than Olist. If Colab dies, sample 100,000 rows and continue.
9. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.

Good luck.